

Rapport TP9 : les Structures de Données

Introduction

a) Définition et Terminologie

Structure de données: c'est une manière de stocker et organiser les données dans un ordinateur de manière à ce qu'il soit utilisé de façon optimale.

structures de données abstraites : Elles définissent les données utilisées et les opérations effectuées sur ces données.

structure de donnée concrète : Ce sont des implémentations des structures de données abstraites

b) Caractérisation de cas de type de structures et différenciations: Cas des Listes

Les listes sont des structure de donnée abstraite caractériser comme suit :

- Elle gère tout type de donnée
- Sur ces données peuvent s'appliquer les opérations suivantes : Stockage, Lecture et Modifications
- Quelques implémentations ou structures concrètes associées : Array(Tableau), Linked List, doubled linked list etc.

Lors de notre étude, nous allons nous attarder sur les structures de données concrètes, en ressortant pour chacun :

- une vue logique de la structure de donnée
- les opérations possible sur la structure de donnée
- le coût des opérations : **complexité temporel** (un ordre de grandeur du temps mis pour réaliser l'opération) et la **complexité spatiale** (une ordre de grandeur de la mémoire occupé pendant ces opérations)

1. Tableau (Array)

Définition

Collection d'éléments de même type stockés dans des emplacements mémoire contigus, accessibles par un index.

Cas d'utilisation

-  Accès de manière aléatoire fréquent(les Tables sont très recommandé si ces opérations sont très fréquente sur vos structure de données et qu'en plus la taille de votre structure est fixe et connue à l'avance.)
 -  Taille fixe connue à l'avance
 -  Opérations mathématiques vectorielles

- ✗ Insertions/suppressions fréquentes (si ces opérations sont fréquentes sur vos structures de données, il n'est pas conseillé d'utiliser les Tableaux car le coût des opérations est moins favorable par rapport à certaines autres structures de données existantes)

Complexité Temporelle des opérations

- Accès $O(1)$: ici on a un temps constant proportionnel à la durée d'exécution d'une instructions,
pour une complexité temporelle $O(1)$ on peut par exemple avoir un temps d'exécution $T = C \times 1U = C$ ou C est une constante et $1U$ (pris comme base d'unité temporelle) est le temps mis pour exécuter par exemple une opération logique (addition, soustraction etc.) par la machine, pour un temps $T = C \times 1U = C$ on a donc une complexité temporelle $O(1)$
- Insertion $O(n)$, ici on a un temps d'exécution qui peut avoir la forme suivante : $T(n) = An + B$ où A et B sont des constantes
- Recherche $O(n)$, ici on a de même un temps d'exécution qui peut avoir la forme suivante : $T(n) = An + B$ où A et B sont des constantes

Complexité spatiale des opérations (espace mémoire occupé pendant l'exécution de l'opération) :

Pour toutes ces opérations, indépendamment de l'algorithme on peut avoir une complexité spatiale $= O(n)$.

L'espace mémoire occupé a donc la forme suivante : $S(n) = (An + B) \times 1U = An + B$ ou $1U$ (pris comme base d'unité d'espace mémoire) est par exemple la taille de l'espace mémoire nécessaire pour stocker un élément du tableau

Algorithme associé aux différentes opérations

* Accès aux Éléments

```
Fonction Accès_Element(Tableau T, Indice i) : Element
    // Vérifier si l'indice est valide
    Si i >= 0 ET i < Taille(T) Alors
        Retourner T[i]
    Sinon
        Afficher "Erreur : Indice hors limites"
        Retourner Null // Ou une valeur d'erreur appropriée
    Fin Si
Fin Fonction
```

On voit ici que quelque soit la taille du tableau on a un nombre constant d'opérations à effectuer d'où une complexité temporelle $O(1)$. Pour cet algorithme on a maximum 3 instructions effectuées (comparaison, retour de valeur etc.)

* Recherche Séquentielle

```
Fonction Recherche_Sequentielle(Tableau T, Valeur V) : Indice
  Pour i allant de 0 à Taille(T) - 1
    Si T[i] == V Alors
      Retourner i // L'élément est trouvé, retourner son indice
    Fin Si
  Fin Pour
  Retourner -1 // L'élément n'a pas été trouvé
Fin Fonction
```

Ici on constate qu'au pire des cas c'est-à-dire si l'élément se trouve à la dernière position , on aura un temps = n fois le temps associés au instructions d'une boucle d'où la complexité temporelle $O(n)$. mais il existe d'autre algorithme de recherche comme le binary search qui permette d'avoir une complexité temporelle bien moindre de l'ordre de $O(\log n)$

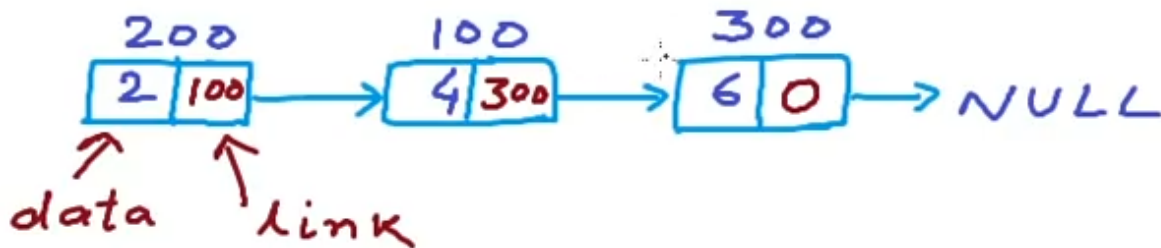
* Insertion Position

```
Procedure Insertion_Position(Tableau T, Indice Pos, Valeur V)
  Si Taille(T) < Taille_Max(T) Alors // Vérifier s'il y a de la place
    // Décaler les éléments vers la droite à partir de la fin jusqu'à
    Pour i allant de Taille(T) - 1 à Pos descendant
      T[i + 1] = T[i]
    Fin Pour
    // Insérer la nouvelle valeur
    T[Pos] = V
    // Mettre à jour la taille (si nécessaire, selon l'implémentation
    Incrementer Taille(T)
  Sinon
    Afficher "Erreur : Tableau plein, impossible d'insérer"
  Fin Si
Fin Procedure
```

2. 🌀 Liste Chaînée (Linked List)

Définition

Liste Chaînée: Ce sont des Collections d'éléments (nœuds) où chaque nœud contient des données et un pointeur vers le nœud suivant. Le pointeur stocke l'adresse du nœuds suivant, ce qui établie le liens entre les deux nœuds



Cas d'utilisation

- ✓ Insertions/suppressions fréquentes
- ✓ Taille dynamique
- ✓ Implémentation de piles/files
- ✗ Accès aléatoire rare

Complexité: Insertion $O(1)$, Accès $O(n)$, Recherche $O(n)$

Algorithme associer aux différentes opérations

```
STRUCTURE Nœud
    données: type
    suivant: pointeur Nœud

DÉBUT ListeChaînée
    tête: pointeur Nœud

MÉTHODE insertionDébut(valeur)
    nouveau = nouveau Nœud(valeur)
    nouveau.suivant = tête
    tête = nouveau

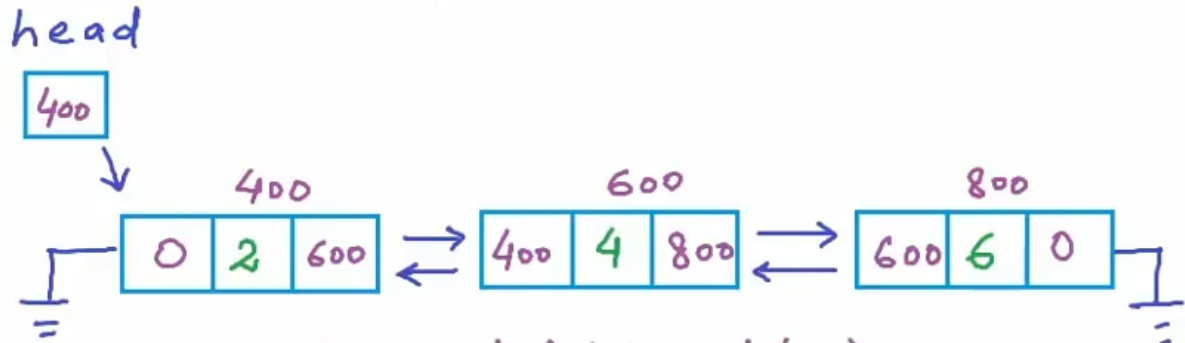
MÉTHODE suppressionDébut()
    SI tête ≠ NULL
        temp = tête
        tête = tête.suivant
        libérer temp

FIN
```

3. Liste Doublement Chaînée (Doubly Linked List)

Définition

Liste Doublement chaînée: Ce sont des Listes chaînées où chaque nœud contient des pointeurs vers le nœud précédent et suivant.



Cas d'utilisation

- ☒ Navigation avant/arrière nécessaire
- ☒ Implémentation de caches (LRU)
- ☒ Systèmes d'historique
- ☒ Mémoire supplémentaire pour les pointeurs

Complexité: Insertion $O(1)$, Suppression $O(1)$, Accès $O(n)$

Algorithme associer aux différentes opérations

```
STRUCTURE NœudDouble
    données: type
    précédent: pointeur NœudDouble
    suivant: pointeur NœudDouble

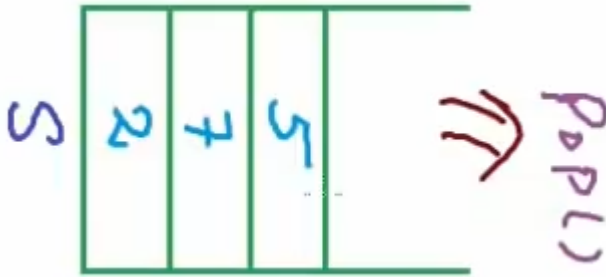
MÉTHODE insertionMilieu(nœudExistant, valeur)
    nouveau = nouveau NœudDouble(valeur)
    nouveau.suivant = nœudExistant.suivant
    nouveau.précédent = nœudExistant
    nœudExistant.suivant.précédent = nouveau
    nœudExistant.suivant = nouveau
```

4. ⚡ Pile (Stack)

Définition

Pile (Stack): Ce sont des Structures de données abstraite souvent appelé structure LIFO (Last In, First Out) où les éléments sont ajoutés et retirés du sommet.

Son implementations concrète peut se faire en utilisant les Arrays ou les Linked lists



Opérations : Insertions(push(x)), supprimer(pop()), obtenir l'élément au top de la stack (Top()), isEmpty() vérifier si la stack est vide

Complexité: Toutes les opérations O(1)

Cas d'utilisation

- ✓ Appels de fonctions (call stack)
- ✓ Annulation/refaire (undo/redo)
- ✓ Évaluation d'expressions
- ✓ Parcours DFS
- ✓ Balanced parenthèses : utiliser par le compilateur pour vérifier les expression mathématique

Algorithme associer aux différentes opérations

```
DÉBUT Pile
  éléments: liste
  sommet: entier = -1

  MÉTHODE push(valeur)
    sommet = sommet + 1
    éléments[sommet] = valeur

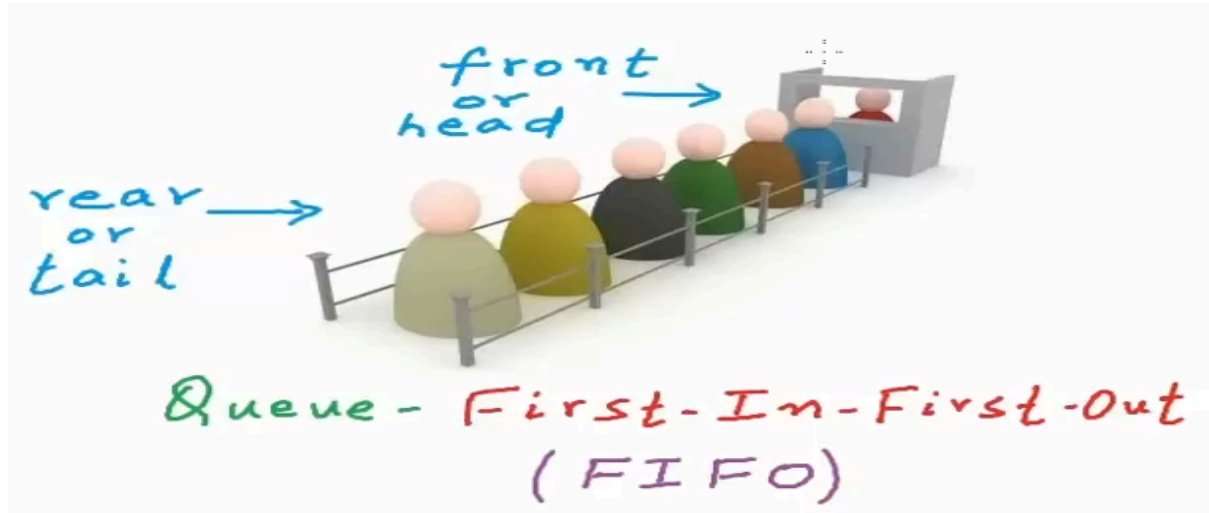
  MÉTHODE pop()
    SI isEmpty() ALORS erreur
    valeur = éléments[sommet]
    sommet = sommet - 1
    RETOURNER valeur

FIN
```

5. 🚩 File (Queue)

Définition

File(Queue) : C'est une structure de données abstraite souvent appelée structure FIFO (First In, First Out) où les éléments sont ajoutés à la fin et retirés du début.



Opérations : Insertions(Enqueue(x) or push(x)), supprimer(dequeue() or pop()), obtenir l'élément au top de la stack (front() or peek()), isEmpty() vérifier si la stack est vide

Complexité: Toutes les opérations $O(1)$

Cas d'utilisation

- ✓ Gestion de processus
- ✓ Traitement de requêtes
- ✓ Buffers d'E/S
- ✓ Recherche BFS

Algorithme associé aux différentes opérations

```
DÉBUT File
    éléments: liste
    tête: entier = 0
    queue: entier = -1

    MÉTHODE Enqueue(valeur)
        queue = queue + 1
        éléments[queue] = valeur

    MÉTHODE Dequeue()
        SI estVide() ALORS erreur
        valeur = éléments[tête]
        tête = tête + 1
        RETOURNER valeur

FIN
```

6. 🏆 File Prioritaire (Priority Queue)

Définition

Priority Queue: C'est une File où chaque élément a une priorité, les éléments de plus haute priorité sont servis en premier.

Cas d'utilisation

- ✓ Ordonnancement CPU
- ✓ Systèmes de tri
- ✓ Algorithmes de Dijkstra, Prim
- ✓ Traitement d'événements

Complexité: Insertion $O(\log n)$, Extraction $O(\log n)$

Algorithme associé aux différentes opérations

```
DÉBUT FilePrioritaire
  // Implémentée avec un tas (heap)
  tas: liste

  MÉTHODE insérer(élément, priorité)
    ajouter (élément, priorité) à tas
    réorganiser le tas vers le haut

  MÉTHODE extraireMax()
    max = tas[0]
    tas[0] = dernier élément
    réorganiser le tas vers le bas
    RETOURNER max

FIN
```


7. 🌳 Tas (Heap)

Définition

Heap: C'est un Arbre binaire complet respectant la propriété de tas (tas max/min).

Cas d'utilisation

- ✓ Implémentation files prioritaires
- ✓ Algorithme de tri heapsort
- ✓ Sélection du k-ème élément
- ✗ Pas pour la recherche d'éléments

Complexité: Insertion $O(\log n)$, Extraction $O(\log n)$

Algorithme associer aux différentes opérations

```
DÉBUT TasMax
  données: liste

  MÉTHODE insertion(valeur)
    ajouter valeur à la fin
    index = taille - 1
    TANT QUE index > 0 ET parent(index) < valeur
      échanger avec parent
      index = parent(index)

  MÉTHODE extraireMax()
    max = données[0]
    données[0] = données[dernier]
    supprimer dernier
    réorganiserVersBas(0)
    RETOURNER max

FIN
```

8. 🌐 Table de Hachage (Hash Table/Map/Dictionary)

Définition

Table de Hachage : C'est une structure qui associe des clés à des valeurs via une fonction de hachage.

Cas d'utilisation

- ✓ Recherche rapide par clé
- ✓ Élimination de doublons
- ✓ Cache
- ✗ Pas pour les données ordonnées

Complexité: Insertion $O(1)$, Recherche $O(1)$ en moyenne

Algorithme associer aux différentes opérations

```
DÉFINIR fonctionHachage(clé) → index

DÉBUT TableHachage
    buckets: liste de listes
    capacité: entier

    MÉTHODE insérer(clé, valeur)
        index = fonctionHachage(clé) MOD capacité
        bucket = buckets[index]
        POUR chaque paire dans bucket
            SI paire.clé = clé ALORS
                paire.valeur = valeur
            RETOURNER
        ajouter nouvelle paire (clé, valeur) à bucket

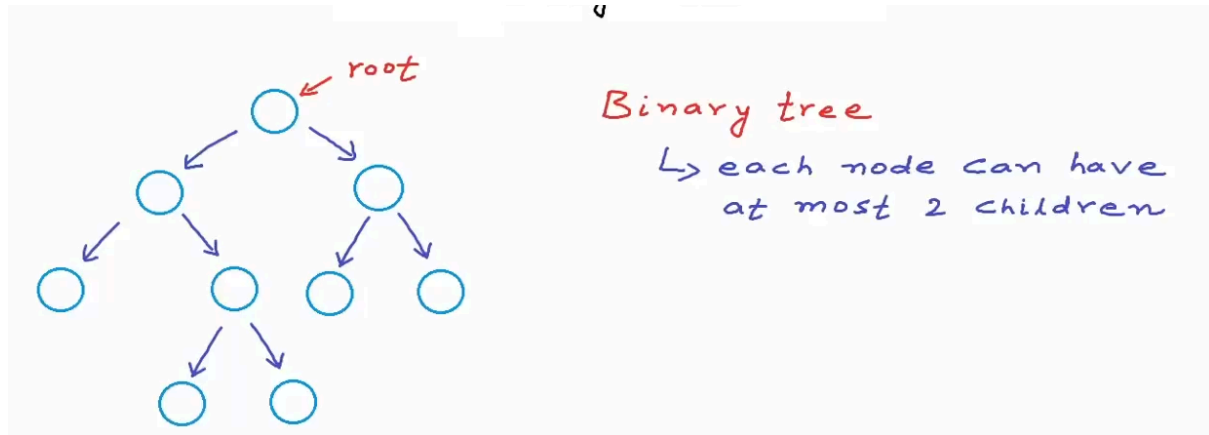
    MÉTHODE obtenir(clé)
        index = fonctionHachage(clé) MOD capacité
        bucket = buckets[index]
        POUR chaque paire dans bucket
            SI paire.clé = clé ALORS
                RETOURNER paire.valeur
        RETOURNER NULL

FIN
```

9. 🌲 Arbre (Tree)

Définition

Tree: C'est une structure hiérarchique avec un nœud racine et des sous-arbres.



Cas d'utilisation

- ✓ Systèmes de fichiers
- ✓ Bases de données
- ✓ DOM XML/HTML
- ✓ Structures organisationnelles

Complexité: Dépend de l'implémentation Arrays ou Linked list

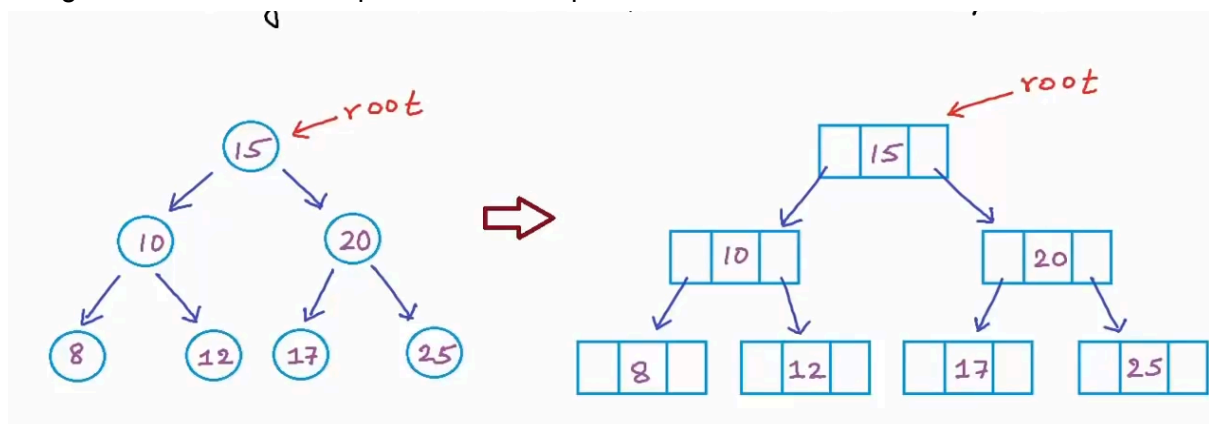
10. 🔍 Arbre Binaire de Recherche (Binary Search Tree - BST)

Définition

BST : c'est un Arbre Binaire ou pour chaque nœud pour lequel on a associé une valeur, tous les éléments du sous-arbre gauche ont des valeurs inférieures à la valeur du nœud de référence et ceux du droit ont des valeurs supérieures.

Arbre Binaire : c'est un arbre ou chaque nœud a au plus 2 enfants

Image d'un BST et sont implementation inspirer des double linked list



Cas d'utilisation

- ✓ Recherche rapide dans données ordonnées
- ✓ Maintenance d'ensemble dynamique trié
- ✗ Peut devenir déséquilibré

Complexité: Recherche $O(\log n)$ en moyenne, $O(n)$ pire cas

Pseudo-algorithme

```
STRUCTURE NœudBST
    données: type
    gauche: pointeur NœudBST
    droit: pointeur NœudBST

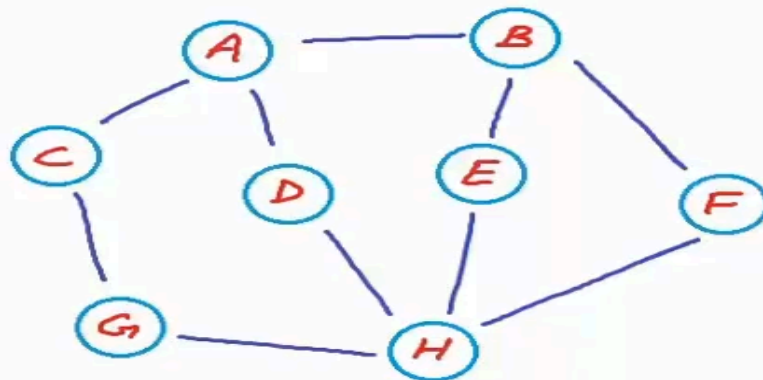
MÉTHODE recherche(racine, clé)
    SI racine = NULL OU racine.données = clé
        RETOURNER racine
    SI clé < racine.données
        RETOURNER recherche(racine.gauche, clé)
    SINON
        RETOURNER recherche(racine.droit, clé)

MÉTHODE insertion(racine, valeur)
    SI racine = NULL
        RETOURNER nouveau NœudBST(valeur)
    SI valeur < racine.données
        racine.gauche = insertion(racine.gauche, valeur)
    SINON
        racine.droit = insertion(racine.droit, valeur)
    RETOURNER racine
```

11. 🌐 Graphe (Graph)

Définition

Graphe: C'est un Ensemble de nœuds connectés par des arêtes (orientées ou non).



Cas d'utilisation

- ✓ Réseaux sociaux
- ✓ Systèmes de navigation
- ✓ Réseaux informatiques
- ✓ Dépendances entre tâches

Complexité: BFS $O(V+E)$, DFS $O(V+E)$

Pseudo-algorithme

```
// Matrice d'adjacence
matrice: tableau 2D[taille][taille]

// Liste d'adjacence
STRUCTURE Graphe
    nœuds: liste
    listeAdjacence: liste de listes

MÉTHODE BFS(départ)
    file = nouvelle File()
    marqués = ensemble vide
    file.enfiler(départ)
    marqués.ajouter(départ)

    TANT QUE file non vide
        courant = file.défiler()
        TRAITER courant
        POUR chaque voisin de courant
            SI voisin non marqué
                file.enfiler(voisin)
                marqués.ajouter(voisin)
```

12. 🎯 Ensemble (Set)

Définition

Set: Collection d'éléments distincts sans ordre particulier.

Cas d'utilisation

- ✓ Vérification d'appartenance
- ✓ Opérations ensemblistes (union, intersection)
- ✓ Élimination de doublons
- ✗ Pas d'accès par index

Complexité: Ajout $O(1)$, Recherche $O(1)$ en moyenne

Pseudo-algorithme

```
// Implémentation avec table de hachage
DÉBUT Ensemble
    table: TableHachage

    MÉTHODE ajouter(élément)
        table.insérer(élément, vrai)

    MÉTHODE contient(élément)
        RETOURNER table.obtenir(élément) ≠ NULL

    MÉTHODE union(otherSet)
        résultat = nouvel Ensemble()
        POUR chaque élément dans this
            résultat.ajouter(élément)
        POUR chaque élément dans otherSet
            résultat.ajouter(élément)
        RETOURNER résultat
FIN
```

13. 🌲 Arbre AVL

Définition

Arbre AVL: C'est un Arbre binaire de recherche auto-équilibré où la hauteur des sous-arbres diffère au plus de 1.

Cas d'utilisation

- ✓ Quand pire cas $O(\log n)$ garanti
- ✓ Bases de données en mémoire
- ✓ Dictionnaires
- ✗ Plus complexe qu'un BST simple

Complexité: Toutes opérations $O(\log n)$

Pseudo-algorithme

```
MÉTHODE équilibrer(nœud)
    équilibre = hauteur(gauche) - hauteur(droit)
    SI équilibre > 1 // Gauche lourd
        SI hauteur(gauche.gauche) >= hauteur(gauche.droit)
            rotationDroite(nœud)
        SINON
            rotationGaucheDroite(nœud)
    SI équilibre < -1 // Droit lourd
        SI hauteur(droit.droit) >= hauteur(droit.gauche)
            rotationGauche(nœud)
        SINON
            rotationDroiteGauche(nœud)
```

14. Arbre Rouge-Noir

Définition

Arbre Rouge-Noir: C'est un Arbre binaire de recherche auto-équilibré avec propriétés de coloration.

Cas d'utilisation

-  Implémentation de TreeMap/TreeSet
-  Systèmes de fichiers
-  Bases de données
-  Programmation concurrente

Propriétés

1. Chaque nœud est rouge ou noir
2. La racine est noire
3. Les feuilles (NULL) sont noires
4. Un nœud rouge a deux enfants noirs
5. Tous chemins d'un nœud à ses feuilles ont même nombre de nœuds noirs

Complexité: Toutes opérations $O(\log n)$

Annexe 1 : Tableau comparatif des complexité



Tableau Comparatif des Complexités

Structure	Accès	Recherche	Insertion	Suppression
Tableau	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Liste Chaînée	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Pile	$O(1)^*$	$O(n)$	$O(1)$	$O(1)$
File	$O(1)^*$	$O(n)$	$O(1)$	$O(1)$
Table Hachage	$O(1)$	$O(1)$	$O(1)$	$O(1)$
BST	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

Annexe 2 : Guide de Sélection

Pour accès rapide par index

-  Tableau
-  Tableau Dynamique

Pour insertions/suppressions fréquentes

-  Liste Chaînée
-  Liste Doublement Chaînée

Pour recherche rapide par clé

-  Table de Hachage
-  Arbre de Recherche

Pour données ordonnées

-  Arbre Binaire de Recherche
-  Tas

Pour relations complexes

-  Graphe

Pour gestion FIFO/LIFO

-  Pile/File

Annexe 3 : Résultats de compilation associé au différend cas d'usage des structures de données

```
• faris@faris-Precision-T1700:~/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource$ cmake .
-- The C compiler identification is GNU 11.4.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/faris/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource
• faris@faris-Precision-T1700:~/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource$ make run
make: *** Aucune règle pour fabriquer la cible « run ». Arrêt.
• faris@faris-Precision-T1700:~/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource$ make
[ 3%] Building C object CMakeFiles/datastructures.dir/src/array.c.o
[ 7%] Building C object CMakeFiles/datastructures.dir/src/linked_list.c.o
[ 11%] Building C object CMakeFiles/datastructures.dir/src/doubly_linked_list.c.o
[ 15%] Building C object CMakeFiles/datastructures.dir/src/stack.c.o
[ 19%] Building C object CMakeFiles/datastructures.dir/src/queue.c.o
[ 23%] Building C object CMakeFiles/datastructures.dir/src/priority_queue.c.o
[ 26%] Building C object CMakeFiles/datastructures.dir/src/bst.c.o
[ 30%] Building C object CMakeFiles/datastructures.dir/src/rbtree.c.o
/home/faris/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource/src/rbtree.c: In function 'removeRBTree':
/home/faris/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource/src/rbtree.c:171:27: warning: unused parameter 'tree' [-Wunused-parameter]
171 | void removeRBTree(RBTree* tree, int key) {
    |                      ^~~~~~
/home/faris/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource/src/rbtree.c:171:37: warning: unused parameter 'key' [-Wunused-parameter]
171 | void removeRBTree(RBTree* tree, int key) {
    |                               ^~~~~~
[ 34%] Building C object CMakeFiles/datastructures.dir/src/graph.c.o
[ 38%] Building C object CMakeFiles/datastructures.dir/src/hash_table.c.o
[ 42%] Building C object CMakeFiles/datastructures.dir/src/tree.c.o
[ 46%] Building C object CMakeFiles/datastructures.dir/src/main.c.o
[ 50%] Linking C executable datastructures
[ 50%] Built target datastructures
[ 53%] Building C object CMakeFiles/test_structures.dir/src/array.c.o
[ 57%] Building C object CMakeFiles/test_structures.dir/src/linked_list.c.o
[ 61%] Building C object CMakeFiles/test_structures.dir/src/doubly_linked_list.c.o
[ 65%] Building C object CMakeFiles/test_structures.dir/src/stack.c.o
[ 69%] Building C object CMakeFiles/test_structures.dir/src/queue.c.o
[ 73%] Building C object CMakeFiles/test_structures.dir/src/priority_queue.c.o
[ 76%] Building C object CMakeFiles/test_structures.dir/src/bst.c.o
[ 80%] Building C object CMakeFiles/test_structures.dir/src/rbtree.c.o
/home/faris/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource/src/rbtree.c: In function 'removeRBTree':
/home/faris/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource/src/rbtree.c:171:27: warning: unused parameter 'tree' [-Wunused-parameter]
171 | void removeRBTree(RBTree* tree, int key) {
    |                      ^~~~~~
/home/faris/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource/src/rbtree.c:171:37: warning: unused parameter 'key' [-Wunused-parameter]
171 | void removeRBTree(RBTree* tree, int key) {
    |                               ^~~~~~
[ 84%] Building C object CMakeFiles/test_structures.dir/src/graph.c.o
```

```
[100%] Linking C executable test_structures
[100%] Built target test_structures
• faris@faris-Precision-T1700:~/Documents/TPetANNEsEsansCODE/TP9StructureDeDonnee/codesource$ ./datastructures
🔥 COMPLETE DATA STRUCTURES DEMO

=== ARRAY DEMO ===
Array contents: 10 20 30
After set index 1 to 25: 10 25 30
Find value 30 at index: 2

=== LINKED LIST DEMO ===
Front: 5
End: 40
Find 30 at index: 2

=== DOUBLY LINKED LIST DEMO ===
Front: 5
End: 40
After remove front: 10
After remove end: 30

=== STACK DEMO ===
Peek: 30
Pop: 30
Pop: 20
Is empty: no

=== QUEUE DEMO ===
Front: 10
Dequeue: 10
Dequeue: 20
Is empty: no

=== PRIORITY QUEUE DEMO ===
Priority queue: 5 10 20 30

=== BINARY SEARCH TREE DEMO ===
BST in-order: 20 30 40 50 60 70 80
Min: 20, Max: 80
Search 40: found
Search 100: not found
After removing 30: 20 40 50 60 70 80

=== RED-BLACK TREE DEMO ===
RBTree in-order: 20(R) 30(B) 40(R) 50(B) 60(R) 70(B) 80(R)
Search 40: found
Search 100: not found

=== GRAPH DEMO ===
Graph adjacency lists:
```

```
=== STACK DEMO ===
Peek: 30
Pop: 30
Pop: 20
Is empty: no

=== QUEUE DEMO ===
Front: 10
Dequeue: 10
Dequeue: 20
Is empty: no

=== PRIORITY QUEUE DEMO ===
Priority queue: 5 10 20 30

=== BINARY SEARCH TREE DEMO ===
BST in-order: 20 30 40 50 60 70 80
Min: 20, Max: 80
Search 40: found
Search 100: not found
After removing 30: 20 40 50 60 70 80

=== RED-BLACK TREE DEMO ===
RBTree in-order: 20(R) 30(B) 40(R) 50(B) 60(R) 70(B) 80(R)
Search 40: found
Search 100: not found

=== GRAPH DEMO ===
Graph adjacency lists:
Vertex 0: 2 -> 1 -> NULL
Vertex 1: 3 -> 0 -> NULL
Vertex 2: 4 -> 0 -> NULL
Vertex 3: 4 -> 1 -> NULL
Vertex 4: 3 -> 2 -> NULL
BFS from 0: 0 2 1 4 3
DFS from 0: 0 2 4 3 1

=== HASH TABLE DEMO ===
Alice: 25
Bob: 30
Contains David: no
Alice after update: 26

=== BINARY TREE DEMO ===
In-order traversal: 20 30 40 50 60 70 80
Search 40: found
Search 100: not found
```

✓ All demos completed!

[Article on FairIs+Precision-11788](#) [~/Documents/TPetAMNIEYEsanaCODE/TPBStructuresReResponse/codesources](#)

Select Language Mode